Object oriented programming Concepts

oops

Class

- A class is a collection of things which process common similarities.

Ex:-Birds is a class contain common similarities/features like.

- Every birds has two legs
- Every birds has two wings.
- Every birds has some color.
- Every birds can fly etc.

Class (cont..)

- Class is a collection of members and those members are :- data member(variables) and member functions(methods).
- Class is container which contain related data and instruction.
- Class is encapsulated with related information (data) and operation (instruction.)
- Class is module/reusable modules.
- Class is data type /abstract data types

Object

- An object is used to represent the class.
- An object is defined as instance of a class. Ex:- parrot is an object for birds class.
- We allocate data(memory) for class is object.
- Copy of a class is know as the object.

Features of object oriented programming

Any programming language to become as object oriented should follow (satisfy) the following feature a part from the class and Object.

- Encapsulation
- Polymorphism
- Inheritance
- Abstraction

Encapsulation

- Encapsulation is mechanism that associated the code and data .it manipulates and keeps the safe from external interfaces and misuse .
- Wrapping of data and code is called encapsulation.
- It is a process of Binding the member variable of a class along with member function.
- Encapsulation can be implemented with the help of object and also access modifiers like private, public and protected etc.

polymorphism

- It derived from the Greek word, where poly means many morph mean faces/behaviors.
- Same function/operator will show different behaviors when passed different type of values or different number of values.
- Changing the behaviors an output of an entity with the change in the input is know as polymorphism

Two types of polymorphism

- Static polymorphism/compile time polymorphism/early binding.
- Dynamic polymorphism/ run time polymorphism / late binding.
- Static polymorphism is achieved using (i) function overloading (ii) operator overloading.
- Dynamic polymorphisms achieved by using :- function overloading

Inheritance

- As per inheritance the member defined in a class can be consume from other class if we establish a parent child relation between the class. Which gives you reusability of code.
- This is a process of creating a new class from already existing class.
- In inheritance process existing class is know as Parent/Base class, newly created class is know as Derived/Child class.

Inheritance (cont..)

- In inheritance process, child class will get all the features of parent/base class.
- Main purpose of Inheritance is code re-usability and providing additional functionality / enhancement.

There are two type of inheritance:-

(i) Single inheritance

(ii) Multiple inheritance

Abstraction

- It is all about hiding the complexity and providing a set of interface to consume the functionality.
- It is a process Hiding the implementation but providing the service.

(i) Data abstraction

(ii) Functional abstraction

Class in C#.net

- In c# .net a class is a user defined data type and is known as reference type.
- A class can contain following members in c#.net.

(i) Data fields

(ii) Functions

(iii) Constructors

(iv) Destructors

(v) Properties

(vi) Indexers

(vii) Events

Diagrammatic representation of class

General notation in UML

Class Name
Attributes
Function

In C# .net

Class Name
Data fields Properties indexers
Constructors Destructors Functions Events

Data fields

- It is used to store the data related to the class.
- Except data fields no other member of a class can store the data.
- To declare any data Field ,we use the following:- syntax

Access modifier datatype data field Name=[initializing value]

Ex:-public int Empid; string Ename; Double salary

Data fields (cont..)

- Access Modifiers:- These are used to define the scope of a class and its member i.e when can access it and when can't .
- C# support 5 access modifiers :-

(i) Private

(ii) Internal

(iii) Protected

(iv) Public

(v) Protected internal

Default accessibility of a class is internal

Functions

- Any programming language will have methods
- A method is a re-usable piece of code which can be called again and again and used to perform required tasks.
- In any programming language a method can be of two types.

(i) Procedures (main)

(ii) functions

Function (cont..)

- A procedures is a method which doesn't return any value to the calling place.
- A function is a method which always will return some value to the calling place, mentioning **return type is compulsory** if function **does not** return any value then mention return type as **void.**
- **Note** :-c# .net will support only functions and doesn’t support procedures, where as vb.net will support both function and procedures.

- Program to create and to consume ex
- To access any member of a class, with the help of object we use **.** (dot) operator known as Member Access operator like Objectname.member Name
- when an object to a class is created, runtime will allot an memory for the members of a class and reference will be maintain from the object, this will be done by new Keyword/operator

- If new keyword is not used, then memory will not be allotted to the data fields and no reference will be maintained from the object and raises an error as Null reference Errors.
- this keyword: this is a keyword used to access the current class in the same class.
- This is an object for the current class in the same class.

How Encapsulation is Implemented

- Encapsulation(ex:-class salary)
- Start->program->visual studio 2010->file->new-> project-> select visual c#-> console application->type application name ex:-(ClsExample)-> select your own location where you want to save ->ok
- Rename the program.cs /class name:- Go to solution explorer->program.cs-> right click on program.cs->rename ->new name(ex:-ClsSalary.cs)

Cont..

- In the above example when obj1.GetSalData() function is called obj1 reference will be copied on to this keyword / obj1 referenced data fields will be bound to GetSalData() function.
- Similarly for DisplaySalData() function also i.e. when ever any function is called with any object of the class that object referenced data fields(member variables) will be bound to the respective Member function, this feature we call as **Encapsulation**.

New keyword doing 3 thins

- Read the classes:-where in this process it identify each and every member that is defined in the class.
- Invokes the constructor
- Allocate the memory required.

constructor

- A constructor is a member method of a class which is invoked automatically when an object to the class is created.
- It’s a special method present in a class which represented for initializing the variable of your class.
- A constructor name should be same name as class name.
- A constructor does not have any return type even void also

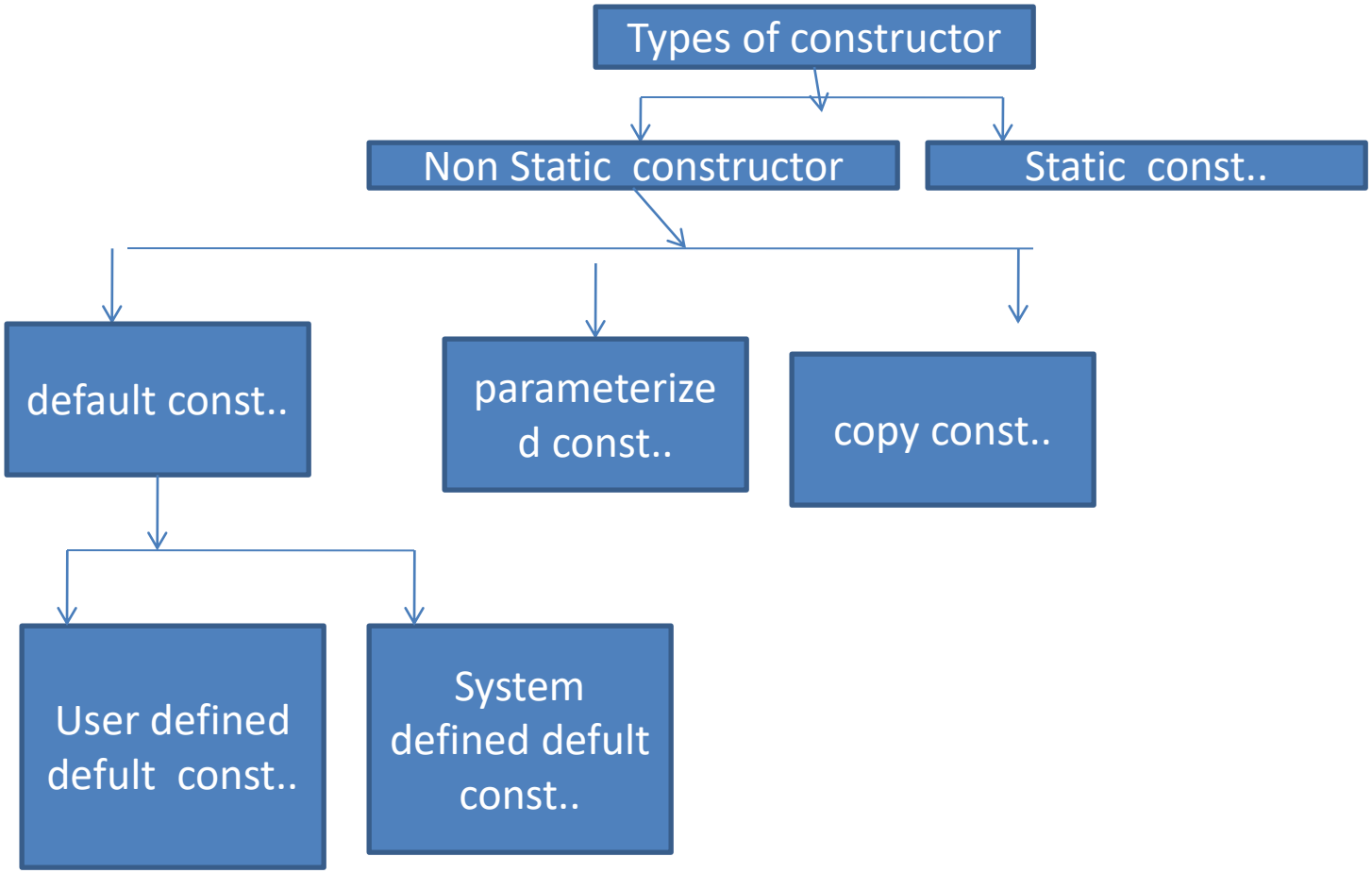
Constructor (cont..)

- A constructor is used to keep something ready for the object when it is created.
- A constructor is used to initialize require values in to the data fields of the class or to pass the required value in to the data fields or to make required valued is establishing connection to data case, keeping the file open to perform some operation etc,when an object is created
- Class contain more then one constructor.

Constructor (cont..)

- A class can not execute if at all it doesn't contain a constructor in it. wherever the compiler compile a class it verifies the programmer defined a constructor in a class or not. if not defined then the compiler will be defined implicitly to met the class to execute.
- Different types of constructor:-

Constructor (cont..)



Constructor syntax

[<modifier>]<Name>([<param define's>])

{

statement;

}

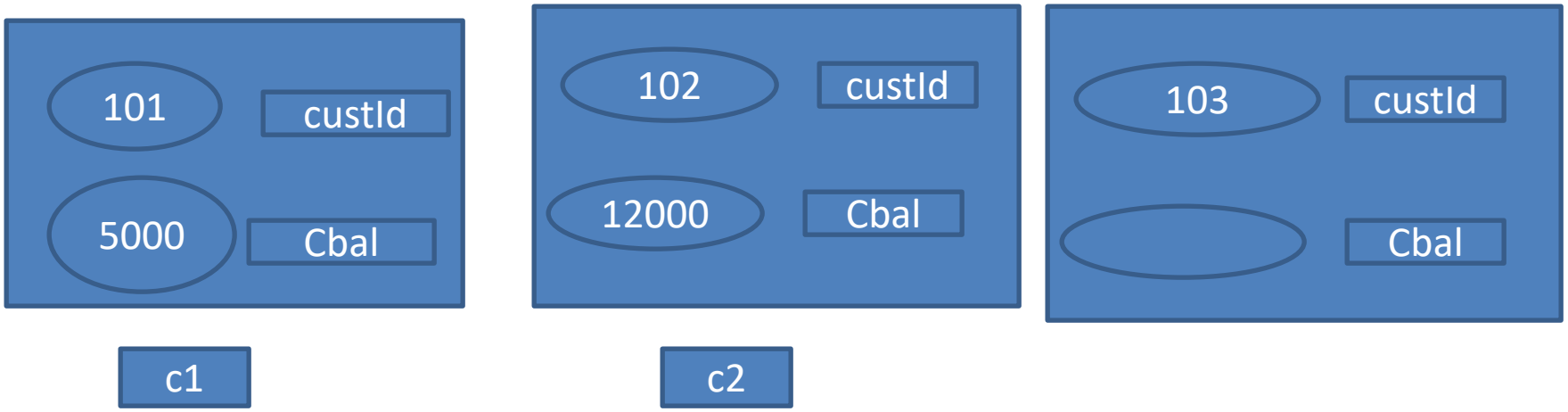
Constructor Demo (default constructor)

Parameterized constructor

Customers table

CustId	Cname	Cbal
101	Xxx	5000
102	Yyy	12000
103	Zzz	15000

```
Customer c1=new Customer(101)
Customer c2=new Customer(102)
```



Customers case study

```
Class customer
{
    double bal;
    int custId;
    static int count=0;
Customer (int custId)
{
    count +=1;
This.custId=custId;
Bal=<get the bal fro db for
    given custId>;
}
Double GetBal()
{
    return bal;
}
Void withdraw(double amt)
{
    bal-=amt;
<update the bal to db for given
    custId>
}
    void Deposite(double amt)
{
    bal+=amt;
<update the db from given custId>
}}
```

User defined default constructor

- This constructor is created by the programmer.
- This constructor doesn't accept any arguments or parameters.
- In General this constructor is used to initialize required values in to the data fields.
- But there is no restriction to write particular code into the constructor. Ex::-program to work with user defined default constructor. (create new project..for it)

System defined default constructor

- If there is no constructor define within a class then ,system will create its own constructor and will assign default value into the data fields
- This constructor created by the system is known as system defined default constructor.
- When as object to the class is created first system will search for a constructor with in the class, if there is no constructor available with in the class, system will create its own constructor and will assign default values in to the fields.(create new project for this default constructor)

Disadvantage of default constructor

- While initializing the data any no of object we may create for a class, **all the object will store same data** in their referenced data fields.
- To over come this drawback ,we use parameterized constructor.

Parameterized constructor

- A parameterized constructor accepts arguments to store the values in to the data fields.
- Using parameterized constructor we can store different set of values into different objects created to the class.
- Create a new application(new project).named ex:-CAP constructor

More then one constructor within a class

- From the above example we can conclude that:-
- A class can contain more then one constructor
- A constructor can be overloaded for different no.of arrguments.

Copy constructor

- This constructor is used to copy the data of an exiting object in to newly created object.
- To copy constructor we need to pass argument that belong to our class data type.

Static constructor

- Static constructor is used to initialize static data fields.
- Static data fields will have maintained only one instance for any no.of objects created to the class.
- To make any member of a class static, use static keyword
- Static members are not accessible with object rather, we need to access with class name only.

- There can be only one static constructor in the class.
- The static constructor should be without in the class.
- It can only access the static members of the class.
- There should be no access modifier in static constructor definition.
- If a class is static, then we can’t create object for static
- Static constructor will be invoked only once i.e 1st object created for the class from second object onwards static constructor will not be called.

Class member

- A class is a collection of member like variable, methods ,constructors etc. These member are categorized into 2types.

(i) instance/non static member

(ii) Static member

- The member of a class which do require the object of a class for initialization or execution are refereed as non static members.
- The member of the class which do not require the object of class for initialization and execution are know as static member

Instance variable and static variables

- Variable declare using static modifier or static block, a variable declare is static variable rest of all are instance variable only. Ex:- int x=100;// Instance

```
Static int y=200;//static
Static void main()//static method
{
   int z =300;
}
```

- **Static variable** are initialization when execution of class which is one and only one time where as non static variable get initialization only after creating the object of the class as well as the each time the object created.
- To invoke a instance member we used object of the class where as to invoke a static member we use name of class.
- We use instance variable whenever the changes of one should not reflecting the other ex:bal in our previous example…

But if at all changes of one has to reflect the other we need to go for static variable ex:- conunt in our pervious class.

- Constant variables:- A variable which holds a value that can't be changed is know as a constant. These variable behave same as a static variable where as a static variable can be modified but constant can’t be modified.
- Read-only variables:- these variable are similar to constant but these are going to be separate copy for each object of the class. where as constant is a single copy to the complete class.

- A constant variable has to be initialized at the time of its declaration only where as read only variable can be initialized either at the time of declaration as well as under construction also.
- The behavior of read only is similar to instance variable where a separate copy will be maintain of each object but instance can be modified were as read only can't be modified.
- Ex of static and instance member

Instance method vs static method

- A method declare by using static modifier is static method rest of all are instance method only.
- While write in static method make sure we are not consuming any instance member of the class directly because non static member can refer from static block only with the object of class in which think are present. but the reverse is possible directly. ex

Static constructor & instance constructor

- A constructor declare using static modifier is a static constructor rest of all are instance constructor only.
- Static constructor initialized static variable and instance variable initialized instance variable.
- A static constructor is the first block of code which execute under a class will be executed one and only one time. Where as instance constructor are called only when the object of class is created as well as each time the object is created.

- Static constructor can’t be parameterized because sending value to it is not possible as we never call then directly. Where as instance constructor can be parameterized.